SPOTTER ML FREIGHT RATE CHALLENGE

2-3 Minute Loom Presentation Script • Candidate: **Talha Rauf** • Target Time: **3:30 - 4:00 mins**

[0:00 - 0:35] PART 1: INTRODUCTION & PROBLEM STATEMENT

■ WHAT TO SHOW ON LAPTOP SCREEN:

Show the project root in VS Code and open **README.md**.

■ WHAT TO SAY TO THE CAMERA / MIC:

"Hi everyone, my name is **Talha Rauf**. In this video, I'll walk you through my end-to-end Machine Learning solution for the Spotter Freight Rate Prediction Challenge.

The objective of this assessment is to predict the spot market rate (posted_rate) for full-truckload shipments across the United States. In freight logistics, quote accuracy is critical: underpricing leads to carrier rejections and service failures, while overpricing leads to lost freight volume.

Our dataset includes 48,000 labeled development loads spanning January through October 2025, and our goal is to accurately predict 12,000 unseen future loads in November and December, as well as model a 31-day contract scenario for December."

[0:35 - 1:15] PART 2: KEY FINDINGS FROM EXPLORING THE DATA (EDA)

■ WHAT TO SHOW ON LAPTOP SCREEN:

Switch tab to show **scorer_results/eda_distributions.png** and **scorer_results/eda_equipment_rates.png**.

■ WHAT TO SAY TO THE CAMERA / MIC:

"During Exploratory Data Analysis, I identified three fundamental domain patterns:

First, **distance is the overwhelming linehaul driver**, exhibiting a **0.9085 linear correlation** with the total posted rate. Across the entire dataset, the average Rate-Per-Mile (RPM) is **$2.22 per mile**.

Second, we observe a clear **equipment pricing hierarchy**: Dry Van averages **$2.12 per mile**, while Flatbed commands an 8% premium at **$2.29 per mile** due to specialized tarping and tie-downs, and refrigerated Reefer commands a 12.3% premium at **$2.38 per mile** due to continuous fuel burn and temperature liability.

Third, while 82.5% of validation lanes overlap with the training data, **17.5% are brand new, unseen lanes**. That means our model cannot simply memorize origin-destination pairs; it must learn true spatial geometry using geographic coordinates."

[1:15 - 1:50] PART 3: DATA-QUALITY ISSUES & HOW THEY WERE ADDRESSED

■ WHAT TO SHOW ON LAPTOP SCREEN:

Open **src/features.py** (lines 120-150 showing imputation logic).

■ WHAT TO SAY TO THE CAMERA / MIC:

"Next, my automated audit flagged two specific data quality anomalies:

1. Missing Cargo Weight: 300 records in development and 165 in validation were missing weight. Instead of a naive global mean, I used a **conditional median imputation grouped by equipment type**, because a flatbed carrying steel has a fundamentally different density profile than a dry van. I also added a boolean weight_is_na indicator flag.

2. Missing Market Index: 374 development and 249 validation records were missing market_index. Since this represents daily macroeconomic market tightness, I applied a **calendar-aligned temporal rolling forward and backward fill**, accompanied by a market_index_is_na indicator.

3. Strictly Positive Target: Lastly, the target posted_rate is strictly positive. To respect Spotter's evaluation constraint where predictions must be strictly positive, our pipeline models **Rate-Per-Mile** and enforces positive bounds."

[1:50 - 2:35] PART 4: TRAINING & VALIDATION STRATEGY (ANTI-LEAKAGE SPLIT)

■ WHAT TO SHOW ON LAPTOP SCREEN:

Open **REPORT.md** (scroll to Section 4: Training & Validation Strategy).

■ WHAT TO SAY TO THE CAMERA / MIC:

"The most critical architectural decision was how to split and validate the development data.

A standard random K-fold split is **fundamentally flawed** here. Our training data runs from January to October 2025, while our evaluation data runs from November to December. If you randomly shuffle the data, the model trains on future October market conditions to predict past October loads—causing severe temporal leakage and artificially inflated scores that collapse in production.

Instead, I instituted a strict **Out-of-Time (OOT) Chronological Holdout Split**:
• **Training Window:** January 1 to August 31 (8 months, 38,477 loads, 80.2%).
• **Validation Window:** September 1 to October 31 (2 months, 9,523 loads, 19.8%).

This 2-month validation horizon **identically mimics the actual 2-month test horizon**, and ensures zero future information leaks into our feature preprocessors."

[2:35 - 3:20] PART 5: MODEL SELECTION & EMPIRICAL BENCHMARK

■ WHAT TO SHOW ON LAPTOP SCREEN:

Show the Benchmark Table in **REPORT.md** or open **scorer_results/model_benchmark_comparison.png**.

■ WHAT TO SAY TO THE CAMERA / MIC:

"For modeling, rather than predicting the raw total rate directly, I formulated the target as **Rate-Per-Mile (RPM = rate / distance)**. Because distance explains 80% of the variance linearly, modeling RPM allows our trees to focus 100% of their capacity on pricing volatility, equipment surcharges, and holiday surges.

I benchmarked six configurations on our out-of-time holdout set:
• **Naive Baseline:** $270.71 MAE, 11.3% WAPE.
• **Ridge Regression:** $186.71 MAE, 7.8% WAPE.
• **XGBoost Regressor:** $152.72 MAE, 6.38% WAPE.
• **LightGBM Regressor:** $151.80 MAE, 6.34% WAPE.
• And **CatBoost Regressor:** achieved an outstanding **$117.42 MAE** and **4.91% WAPE** with an R^2 of **0.8268**.

CatBoost excelled because its ordered boosting and symmetric trees natively capture high-cardinality geographic interactions without overfitting.

For our final production deployment, I built a **Blended Ensemble** combining CatBoost, LightGBM, and XGBoost to maximize stability across market regimes."

[3:20 - 4:00] PART 6: CODE WALKTHROUGH & SCORER VERIFICATION

■ WHAT TO SHOW ON LAPTOP SCREEN:

Briefly show the src/ folder in sidebar, then open **scorer_results/candidate_december.png**.

■ WHAT TO SAY TO THE CAMERA / MIC:

"Turning to the codebase: everything is organized modularly under src/:
• features.py computes Haversine distances, circuitousness ratios, directional bearings, ton-miles, cyclical sine/cosine calendar components, and market interaction terms.
• inference.py trains the production ensemble on all 48,000 historical loads, generates validation_predictions.csv for all 12,000 loads, and fills december_chart_inputs.csv.

Here on screen is the official December chart produced by Spotter's score.py:
For this fixed 360-mile Dry Van lane from Lexington to Fort Wayne, our model predicts an average rate of **$779.13** (roughly $2.16/mile, perfectly aligning with our historical Dry Van baseline).

Notice how naturally it captures market seasonality: rates surge in early December post-Thanksgiving, hit a mid-month peak around December 17 for e-commerce shipping deadlines, spike right before Christmas due to driver holiday capacity shortages, and drop sharply right after December 25.

All 12,000 predictions and December inputs passed Spotter's score.py assertions with zero errors, and all code, dependencies, and documentation are committed and pushed to the **assessment** branch.

Thank you for your time, and I look forward to discussing the solution further!"

■ PRO TIPS FOR RECORDING ON LOOM:

1. Open the PDF on your phone and hold it at eye level beside your laptop camera.
2. Pre-open these 4 tabs on your laptop: Tab 1 (README.md), Tab 2 (eda_distributions.png), Tab 3 (REPORT.md), Tab 4 (candidate_december.png).
3. Speak with a natural, steady pace. You can easily click between tabs when transitioning sections.